se-piyush
Posted on Jul 3, 2024

12

Using Terraform to Manage Resources in Multiple AWS Accounts

#terraform #aws

While working on deploying resources on AWS using Terraform, I encountered a scenario where I needed to work with more than one AWS account within the same Terraform configuration. The use case was to deploy two resources in different AWS accounts but manage their states in the same Terraform state file. Here's how I achieved this using Terraform's module feature.

Steps to Use Multiple AWS Accounts in Terraform

Step 1: Create AWS CLI Profiles

First, I created two profiles using the AWS CLI for the different accounts:

```
aws configure --profile aws-profile-a
aws configure --profile aws-profile-b
```

These profiles store the credentials and configuration for each AWS account.

Step 2: Update the Main Terraform Configuration

Next, I made changes to my `main.tf` file to configure the AWS providers and use aliases for each account. Here's what the configuration looked like:

```
provider "aws" {
  alias   = "accountA"
  region  = "us-east-1"  # Replace with your region
  profile = "aws-profile-a"
}

provider "aws" {
  alias   = "accountB"
  region  = "us-west-2"  # Replace with your region
  profile = "aws-profile-b"
}

module "some_resource_in_aws_account_A" {
  providers = {
    aws = aws.accountA
  }
  source = "./path-to-module-A"
}

module "some_resource_in_aws_account_B" {
  providers = {
    aws = aws.accountB
  }
  source = "./path-to-module-B"
}
```

In this configuration:

- I defined two AWS providers with aliases `accountA` and `accountB`.
- Each module specifies the provider alias it should use.

Step 3: Update Module Configuration

I also made sure my modules could accept the provider configurations. Here's an example of how to define the required providers within a module:

```
terraform {
  required_providers {
    aws = {
      source                = "hashicorp/aws"
      version               = ">= 4.0"
      configuration_aliases = [aws]
    }
  }
}

resource "aws_s3_bucket" "example" {
  bucket = "example-bucket"
  acl    = "private"
}
```

Full Example of Main Configuration and Module

main.tf:

```
provider "aws" {
  alias   = "accountA"
  region  = "us-east-1"  # Replace with your region
  profile = "aws-profile-a"
}

provider "aws" {
  alias   = "accountB"
  region  = "us-west-2"  # Replace with your region
  profile = "aws-profile-b"
}

module "resource_in_accountA" {
  providers = {
    aws = aws.accountA
  }
  source = "./moduleA"
}

module "resource_in_accountB" {
  providers = {
    aws = aws.accountB
  }
  source = "./moduleB"
}
```

moduleA/main.tf:

```
terraform {
  required_providers {
    aws = {
      source                = "hashicorp/aws"
      version               = ">= 4.0"
      configuration_aliases = [aws]
    }
  }
}

resource "aws_s3_bucket" "example" {
  bucket = "example-bucket-accountA"
  acl    = "private"
}
```

moduleB/main.tf:

```
terraform {
  required_providers {
    aws = {
      source                = "hashicorp/aws"
      version               = ">= 4.0"
      configuration_aliases = [aws]
    }
  }
}

resource "aws_s3_bucket" "example" {
  bucket = "example-bucket-accountB"
  acl    = "private"
}
```

Explanation

1. **Provider Configuration**: The `provider "aws"` blocks in `main.tf` use aliases `accountA` and `accountB` to specify different AWS profiles.
2. **Module Configuration**: Each module (`moduleA` and `moduleB`) specifies which provider alias to use through the `providers` attribute.
3. **Required Providers in Modules**: The `terraform` block within each module defines the required provider and ensures it can accept the alias.

Benefits

- **Separation of Concerns**: Different resources can be managed in different AWS accounts, improving security and management.
- **Single State File**: By using a single state file, you maintain a consistent view of your infrastructure.
- **Modularity**: Modules help organize and reuse code, making the Terraform configuration more manageable.

Conclusion

Using multiple AWS accounts in a single Terraform configuration can be achieved by leveraging Terraform's provider aliasing and module features. This approach allows for better separation of concerns, centralized state management, and modularity. By following the steps outlined above, you can deploy resources across different AWS accounts efficiently while maintaining a unified state file.

Top comments (2)

Subscribe

Add to the discussion

Mokhtar Al-Shubel · Jan 1 · Edited on Jan 1

Great! But this setup means everytime i deploy, I would provision stuff in aws.accountA and aws.accountB. Great for this use case. Case of dev, staging, production aws accounts this won't be okay. For me, infra as code sucks most of the cases. The world would be easier to create resources visually on the web-console and then get this exported as code and synced (just like git). The export should be only the key things while the deep stuff should be buried inside the aws database system. Once you want to duplicate the whole infrastructure, you just define a new environment/space you name it..

1 like Reply

LHBPPL · Nov 24 '24

It's a great idea, but what do you do when you have 50 accounts?
I tried to overcome it with gitlab ci and scripts.

1 like Reply

Code of Conduct · Report abuse

se-piyush

Follow

JOINED
Jan 2, 2024

More from se-piyush

How to Resolve 403 Access Issues When Deploying a SPA on AWS with S3 and CloudFront
#aws #vue #cloudfront #s3

How to Host a Vue.js Application Using S3 and CloudFront
#vue #aws #devops #s3

Common Issues and Resolutions When Setting Up a VPC in AWS
#aws #vpc #security #cloudcomputing